Введение в машинное обучение, Kaggle, неделя 5

Классификация спутниковых снимков

Ансамбль из шести предобученных сетей семейства ResNet

Команда «Хиханьки да хаханьки»

Анна Чиркова, Полина Хадралинова

Задача

По спутниковому снимку 256×256 нужно определить, что на нём. Всего 20 классов: мост, озеро, вокзал, взлётная полоса, теннисный корт и другие.

Часть данных	Снимков	Для чего
train	4 999	обучение
validation	699	проверка
test	8 299	предсказание, часть снимков испорчена

Метрика: accuracy. Итоговая модель должна обучаться не дольше 45 минут.

С чего начали

Модели из лабораторной, accuracy на валидации



Предобученная ResNet-18 обошла все свои сети больше чем на 20 п.п., поэтому дальше мы работали только с предобученными моделями.

Как устроено решение

Снимок

256×256, для двух сетей увеличен до 288×288

→

6 сетей

ResNeXt-101, Wide ResNet-101, 2 × ResNet-152, 2 × ResNet-101

→

Свой угол

каждая сеть видит тестовый снимок повёрнутым или отражённым по-своему

→

Среднее

вероятности 20 классов усредняются, ответ: самый вероятный класс

- все сети предобучены на ImageNet, мы дообучаем последние блоки и новый слой на 20 классов
- при обучении снимки специально портятся так же, как в тесте
- сети учатся одновременно на двух видеокартах Kaggle, по три на каждой
- итоговые модели учатся на train вместе с validation

Код: состав ансамбля

```
BASE = {
    "arch": "resnet152", "weights": "IMAGENET1K_V2",
    "unfreeze": ["layer2", "layer3", "layer4"], "dropout": 0.3,
    "img_size": 256, "batch_size": 64, "epochs": 10, "lr": 5e-4,
    "weight_decay": 1e-4, "label_smoothing": 0.1,
    "p_blur": 0.3, "p_lowres": 0.15, "p_gray": 0.15,
    "p_dark": 0.15, "p_noise": 0.15, "augmix": False, "seed": 42,
}
RX64 = {"arch": "resnext101_64x4d", "weights": "IMAGENET1K_V1",
        "batch_size": 32}
WRN = {"arch": "wide_resnet101_2", "batch_size": 32}
MEMBERS = [
    {**RX64, "seed": 42, "view": 0},
    {**WRN, "seed": 42, "view": 4},
    {"arch": "resnet152", "img_size": 288, "batch_size": 48,
     "seed": 42, "view": 1},
    {"arch": "resnet152", "img_size": 288, "batch_size": 48,
     "seed": 1, "view": 5},
    {"arch": "resnet101", "seed": 42, "view": 2},
    {"arch": "resnet101", "seed": 1, "view": 6},
]
```

BASE: общие настройки для всех шести сетей.

MEMBERS: сами сети. У каждой записано только то, чем она отличается от BASE.

view: под каким углом сеть смотрит на тестовые снимки.

seed: одинаковые сети учатся с разными сидами, чтобы ошибаться по-разному.

week5_solution_v6c.ipynb, словари записаны компактнее, чем в ноутбуке

Код: аугментации

```
def make_train_transform(cfg):
    return transforms.Compose([
        transforms.RandomHorizontalFlip(p=0.5),
        transforms.RandomVerticalFlip(p=0.5),
        transforms.RandomChoice([transforms.RandomRotation((angle, angle))
                                 for angle in (0, 90, 180, 270)]),
        transforms.ColorJitter(brightness=0.2, contrast=0.2),
        transforms.RandomApply([transforms.ColorJitter(brightness=(0.3, 0.7))],
                               p=cfg["p_dark"]),
        transforms.RandomGrayscale(p=cfg["p_gray"]),
        transforms.RandomApply(
            [transforms.GaussianBlur(kernel_size=9, sigma=(0.5, 3.0))],
            p=cfg["p_blur"]),
        transforms.RandomApply([LowRes()], p=cfg["p_lowres"]),
        transforms.ToTensor(),
        transforms.RandomApply([AddNoise()], p=cfg["p_noise"]),
        transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
    ])
```

В тесте есть размытые, шумные, чёрно-белые и тёмные снимки. При обучении мы портим снимки так же, каждое искажение со своей вероятностью из BASE.

LowRes и **AddNoise**: свои классы. Первый уменьшает снимок в 2–4 раза и растягивает обратно, второй добавляет шум.

Повороты и отражения тоже подходят: у снимка со спутника нет верха и низа.

ResNet-18 на валидации: без этих искажений **0.917**, с ними **0.953**.

Строка с AugMix опущена: в итоговом решении он выключен

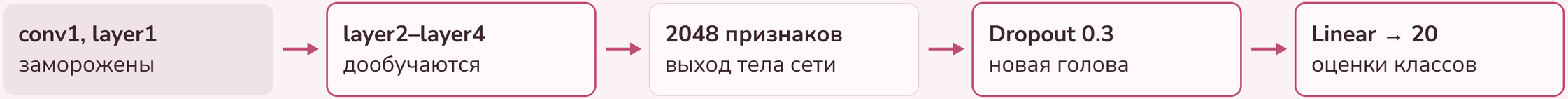
Код: модель

```
def build_model(cfg):
    """Предобученная ResNet: заморожено всё, кроме блоков из cfg["unfreeze"];
    свой классификатор на 20 классов."""
    seed_everything(cfg["seed"])
    backbone = getattr(models, cfg["arch"])(weights=cfg["weights"])
    unfreeze_all = "all" in cfg["unfreeze"]
    for name, param in backbone.named_parameters():
        param.requires_grad = unfreeze_all or \
            name.split(".")[0] in cfg["unfreeze"]
    num_features = backbone.fc.in_features
    backbone.fc = nn.Identity()
    classifier = nn.Sequential(nn.Dropout(cfg["dropout"]),
                               nn.Linear(num_features, NUM_CLASSES))
    return nn.Sequential(backbone, classifier)
```

getattr(models, ...): берём сеть из torchvision по имени и загружаем веса ImageNet.

requires_grad: первые слои заморожены, они уже умеют находить края и текстуры. Учатся только layer2, layer3 и layer4.

fc: последний слой на 1000 классов ImageNet заменяем на Dropout и Linear на 20 классов.



Длинные строки перенесены, чтобы поместиться на слайд

Код: обучение

```
def train(model, train_data, cfg, device):
    ...
    loader = make_loader(train_data, cfg, shuffle=True)
    criterion = nn.CrossEntropyLoss(label_smoothing=cfg["label_smoothing"])
    optimizer = torch.optim.AdamW(
        [p for p in model.parameters() if p.requires_grad],
        lr=cfg["lr"], weight_decay=cfg["weight_decay"])
    scheduler = torch.optim.lr_scheduler.OneCycleLR(
        optimizer, max_lr=cfg["lr"], total_steps=cfg["epochs"] * len(loader))
    for epoch in range(cfg["epochs"]):
        model.train()
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            with torch.autocast(...):
                loss = criterion(model(images), labels)
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()
            scheduler.step()
```

AdamW обновляет только размороженные веса.

OneCycleLR: шаг обучения сначала растёт, потом плавно уменьшается за 10 эпох.

label_smoothing = 0.1: модель не учится быть уверенной на 100 %.

autocast и **scaler**: вычисления в половинной точности, так обучение быстрее.

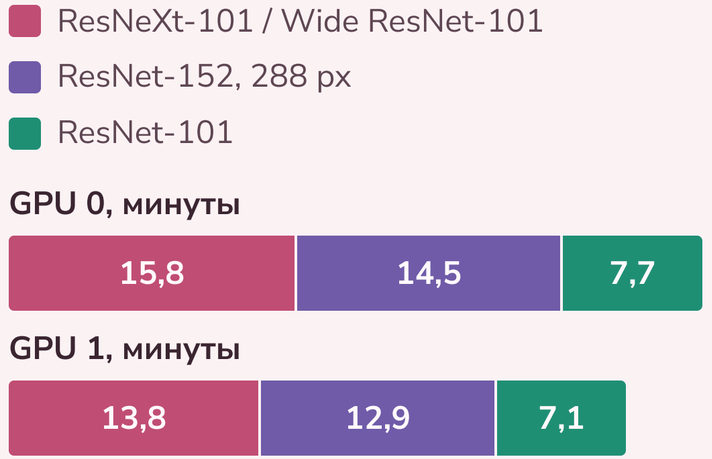
Код сокращён: на месте «...» сиды, перенос модели на видеокарту и настройки половинной точности

Код: обучение на двух видеокартах

```
def run_on_all_gpus(job, items):
    errors = []
    def worker(device, part):
        try:
            for item in part:
                job(item, device)
        except Exception as error:
            errors.append(error)
    threads = [threading.Thread(target=worker,
                                args=(device, items[i::len(GPUS)]))
               for i, device in enumerate(GPUS)]
    for t in threads: t.start()
    for t in threads: t.join()
    if errors:
        raise errors[0]

start_time = time.time()
run_on_all_gpus(train_member, ensemble)
end_time = time.time()
elapsed_minutes = (end_time - start_time) / 60
```

Каждая видеокарта в своём потоке учит свои три сети. **items[i::2]** раздаёт сети по очереди: 1, 3 и 5 на GPU 0, остальные на GPU 1.



Обучение всего ансамбля: **38,0 минуты**.

Код: предсказание

```
def to_view(images, view):
    """Вид картинки: поворот на 90 градусов view % 4 раз
    и, для view ⩾ 4, отражение."""
    images = torch.rot90(images, view % 4, (2, 3))
    return images.flip(3) if view ⩾ 4 else images

def predict_test(member, device):
    member["test_probs"] = predict_probs(
        member["model"], test_u8[member["cfg"]["img_size"]],
        member["cfg"], member["device"])

run_on_all_gpus(predict_test, ensemble)
labels = sum(m["test_probs"] for m in ensemble).argmax(dim=1).numpy()
```

predict_test в ноутбуке записан одной строкой, здесь разбит на три

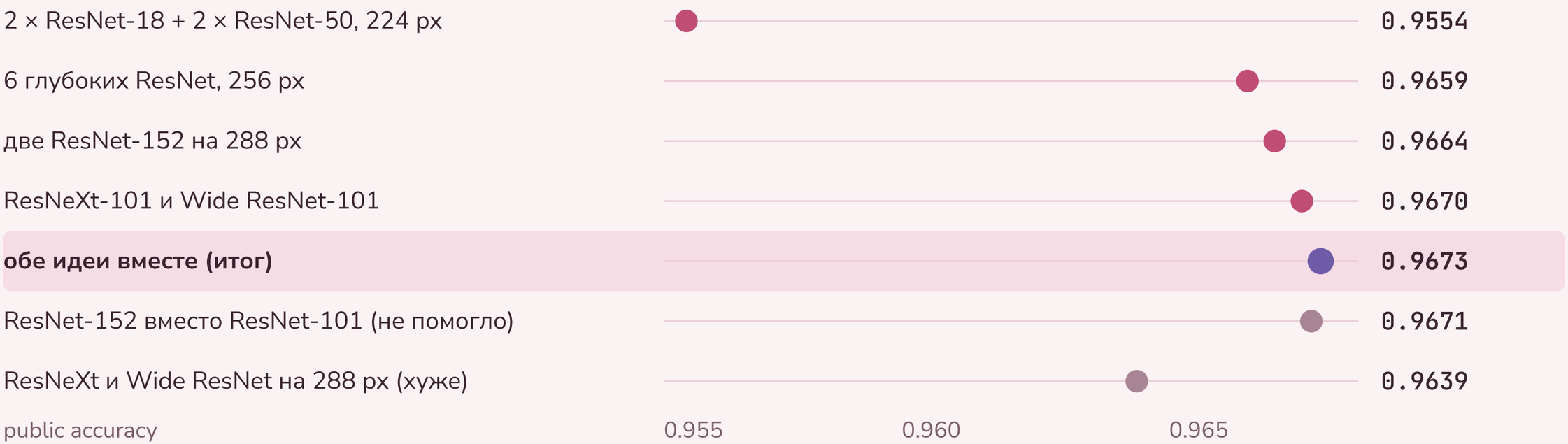
to_view поворачивает снимок на 90° столько раз, сколько view % 4, а при view ≥ 4 ещё отражает. Всего 8 вариантов.

У шести сетей view 0, 4, 1, 5, 2, 6: каждая смотрит на тест по-своему, а проход по тесту у каждой один.

predict_probs даёт вероятности 20 классов. Складываем их по шести сетям, **argmax** выбирает класс с наибольшей суммой.

Весь тест, 8 299 снимков, предсказывается за 127 секунд.

Как менялся результат



На валидации (699 снимков) эти варианты отличались на 1–3 снимка, это в пределах случайного разброса. Поэтому сравнивали по лидерборду: одна идея на одну отправку.

Итоги

7 место
на итоговом лидерборде

0.9642
accuracy, итоговый лидерборд

0.9673
accuracy, public лидерборд

Помогло

- предобученные сети вместо своих
- аугментации, похожие на дефекты теста
- ансамбль из шести сетей на двух видеокартах
- ResNeXt и Wide ResNet в ансамбле
- две ResNet-152 на снимках 288 px

Не помогло

- меньший шаг обучения
- Inception v3 в ансамбле
- ResNet-152 вместо ResNet-101
- ResNeXt и Wide ResNet на 288 px